```
  # ...or pass _any_ formatter class, including a custom one:
  config.add_formatter Fuubar
end
```

This example uses the Fuubar formatter, which is one of the more popular and useful third-party formatters.[7]

As the add_formatter method suggests, you can add multiple formatters, directing each to a different output:

09-configuring-rspec/03/rspec_configure.rb

```
RSpec.configure do |config|
  config.add_formatter 'documentation', $stdout
  config.add_formatter 'html', 'specs.html'
end
```

If you don't call add_formatter or pick a formatter from a command-line option, RSpec will default to the progress formatter. However, you can provide a different default using config.default_formatter:

09-configuring-rspec/03/rspec_configure.rb

```
RSpec.configure do |config|
  config.default_formatter = config.files_to_run.one? ? 'doc' : 'progress'
end
```

With this snippet, RSpec will default to the more verbose documentation formatter if you're running just one spec file, or the progress formatter if you're running multiple files. Regardless, you can override this default by passing a formatter on the command line.

Library Configuration

The most common way to run RSpec is to use rspec-core to run your specs, rspec-expectations to express expected outcomes, and rspec-mocks to provide test doubles. But you don't *have* to use them together. They ship as three separate Ruby gems precisely so that you can swap any of them out. You can even use rspec-mocks or rspec-expectations with another test framework, as we discuss in *Using Parts of RSpec With Other Test Frameworks*, on page 296.

Mocks

The config.mock_with option sets which mock object framework RSpec will use. If you want to use Mocha instead of RSpec mocks, you can do so with the following code:[8]

7. https://jeffkreeftmeijer.com/2010/fuubar-the-instafailing-rspec-progress-bar-formatter/
8. http://gofreerange.com/mocha/docs/